

SECURITY INCIDENT REPORT

Account Takeover Detection & Post-Authentication Anomaly Detection
SecureBank Decoy Portal

Home Lab SOC Simulation

Prepared by: Vignesh Gopal Kalaivani

Environment: Kali Linux (Attacker) → Debian 13 / Apache / Wazuh SIEM (Target + Manager)

Report Date: September 18, 2026

Classification: Lab / Portfolio Use — No Production Data Involved

1. Executive Summary

This report covers a simulated account takeover attack against a decoy online-banking login page ("SecureBank"), built in a self-hosted home lab, and the detection work done to catch it using Wazuh SIEM. The project happened in two stages.

First, a credential-stuffing attack was launched from an attacker machine (Kali Linux) against the fake login endpoint. Custom Wazuh rules were written to catch a burst of failed logins followed by a successful one from the same IP — the clearest sign of a successful takeover.

Second, after a practitioner reviewing the project pointed out a real gap, the detection was extended past the login moment. The original rules only look at authentication events, so a session that logs in cleanly and is then hijacked by someone else would slip through completely. Two new detections were built to close that gap: one for sensitive actions taken without a fresh re-login, and one for a session suddenly being used from a different IP with no new login in between.

Both stages worked. The first produced two confirmed Account Takeover alerts (rule 100104, severity 14), mapped to MITRE ATT&CK T1110 and T1078. The second produced confirmed alerts for the step-up gap (rule 100105) and the session hijack (rule 100106, mapped to T1550.004), after a fair amount of debugging that surfaced nine separate, non-obvious things about how Wazuh actually behaves.

The whole exercise mirrors what Canadian banks are being pushed to do right now. The Canadian Anti-Fraud Centre logged record fraud losses in 2025, and incoming Bank Act amendments will require banks to actively detect and prevent account fraud. This lab is a hands-on demonstration of the kind of behavioral correlation that requirement is asking financial-sector SOC teams to build.

2. Scenario

The decoy login page lives at /securebank/login.php on a Debian 13 box running Apache and PHP. A second endpoint, /securebank/transfer.php, simulates a sensitive banking action (a fund transfer) and became the enforcement point for the later detection work. Two fake customer accounts were set up:

Username	Password	Role
jsmith	Winter2026!	Decoy account #1
mchen	Toronto123	Decoy account #2

Kali (192.168.1.10) ran a credential-stuffing attack against the login page using Hydra's http-post-form module — multiple usernames against multiple passwords, structured like a real leaked-credential dump so the correct password for each account sits somewhere in the middle of the list rather than being tried first. Later, Kali was also used to steal and replay a session cookie against the transfer endpoint, skipping the login flow entirely.

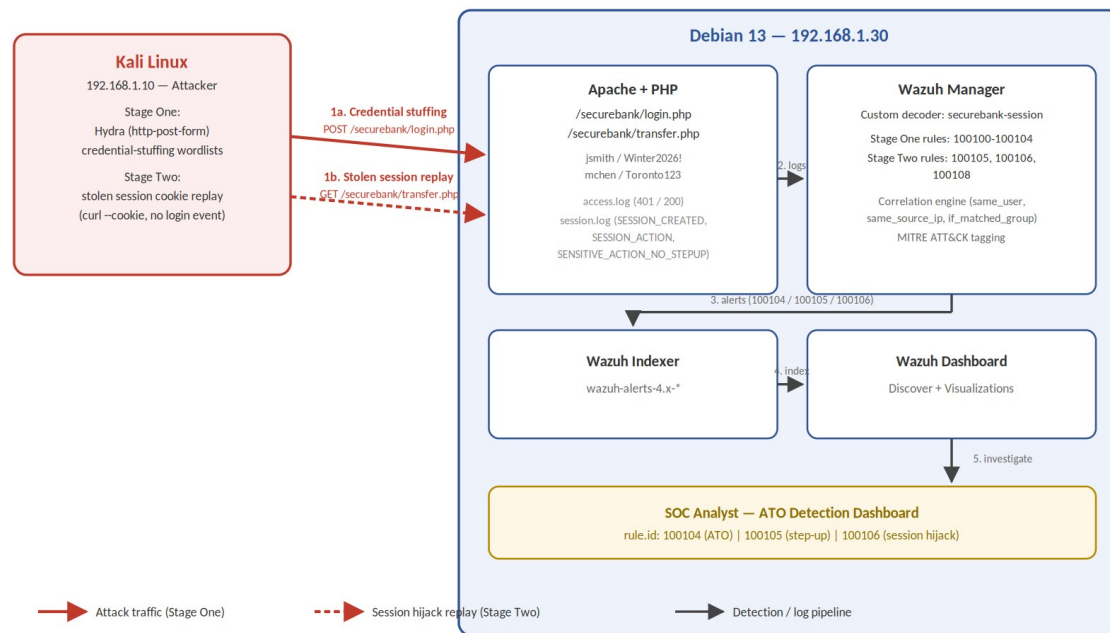
Debian (192.168.1.30) hosts both the decoy app and the full Wazuh stack — manager, indexer, dashboard — reusing the same infrastructure from an earlier SSH brute-force detection project.

3. Architecture & Data Flow

The diagram below shows the lab layout and how data moves from the initial attack through detection to investigation.

ATO & Session Anomaly Detection — Architecture & Data Flow

Host-only network 192.168.1.0/24 — Stage One (login-based) + Stage Two (post-auth, Phase 9)



Project 6 — Account Takeover & Post-Authentication Anomaly Detection Lab | Built on Project 5 infrastructure

Kali sends requests to both endpoints (1). Apache logs every request to access.log, and the app separately logs session activity to a dedicated session.log — both watched by Wazuh as local file sources (2). Custom rules evaluate each event and raise correlated alerts, most importantly rule 100104 for confirmed takeover and rules 100105/100106 for the post-login anomalies (3). The indexer stores and indexes those alerts (4), and the analyst works from the Dashboard's Discover view and the custom ATO Detection Dashboard (5).

4. Attack Execution — Stage One

4.1 Baseline Check

Before running anything automated, a couple of manual requests confirmed the login endpoint behaved as expected:

```
curl -X POST http://192.168.1.30/securebank/login.php -d "username=jsmith&password=wrongpass"
curl -X POST http://192.168.1.30/securebank/login.php -d "username=jsmith&password=Winter2026!"
```

The first came back 401, the second 200 — confirming the login logic worked correctly and giving a clean baseline for what the detection rules would need to catch.

4.2 Credential Stuffing

```
hydra -L usernames.txt -P stuffing_passwords.txt 192.168.1.30 http-post-form \
"/securebank/login.php:username=^USER^&password=^PASS^:Invalid username"
```

Hydra worked through the username/password combinations, using the string "Invalid username" in the response to tell failures from successes. The run produced a clear pattern in the logs — a run of 401s with two 200s mixed in, one for each real account. A quick grep afterward confirmed exactly two successful logins, matching the two accounts that actually existed.

5. Detection Engineering — Stage One

Out of the box, Wazuh's Apache rules have no idea what a login endpoint even is — they lumped this whole attack into a generic "Vulnerability Scanning" bucket. Custom rules were needed to actually recognize what was happening.

5.1 Getting the Logs In

```
<localfile>
  <log_format>apache</log_format>
  <location>/var/log/apache2/access.log</location>
</localfile>
```

This let Wazuh's built-in Apache decoder break each request into fields — source IP, method, URL, status code — without needing anything custom at this stage.

5.2 The Rules

Rule ID	Level	What it does
100100	0	Matches any request to the login endpoint — just a base condition, doesn't alert
100101	5	A single failed login — low priority, everyone fat-fingers a password sometimes
100102	10	6+ failed logins in 60 seconds from one IP — stuffing in progress
100103	3	A successful login
100104	14	A stuffing burst followed by a success from the same IP within 120 seconds — takeover confirmed

Rule 100104 is really the point of the whole first stage. Instead of firing on one event, it waits for two things to line up — a burst of failures and then a success, same source, inside a time window. That combination is a far stronger and quieter signal than either one on its own, and it's basically how real fraud-correlation engines think.

5.3 What Went Wrong (and Why)

- First attempt at rule 100104 used frequency="1" and Wazuh flatly rejected it: "Must be higher than 1." Makes sense once you think about it — you can't correlate one event against itself. frequency="2" wasn't a workaround, it was just the correct number for what the rule was actually trying to say.
- Rule 100103 (successful login) never fired, even though the 200s were clearly sitting in the Apache log. Testing a real log line through wazuh-logtest showed Wazuh was routing 200-status requests through its own built-in rule 31108 ("Ignored URLs"), not 31100, which the custom chain had been built on top of. Wazuh only lets one rule win per log line, so the fix was chaining onto 31108 — the rule actually catching the traffic — instead of the one that seemed right on paper.

6. Detection Engineering — Stage Two

Stage One only looks at the moment of login. A practitioner reviewing the project put the gap plainly: failed attempts, stuffing, impossible travel — all of that shows up in the log signature. The hard case is a session that logs in cleanly and then gets used by someone else, and in a real bank that usually looks like nothing at all. Stage Two builds the two signals a SIEM can actually see in that situation.

6.1 Sensitive Actions Without a Fresh Login

Every login now stamps the session with a creation time. The transfer endpoint checks that timestamp before letting a sensitive action through:

```
$STEPUP_WINDOW_SECONDS = 120;
$page = time() - ($SESSION['login_time'] ?? 0);
if ($page > $STEPUP_WINDOW_SECONDS) {
  // log SENSITIVE_ACTION_NO_STEPUP, return HTTP 403
} else {
  // log SESSION_ACTION, allow the transfer
}
```

Inside the window, it goes through normally. Outside it, the request gets blocked with a 403 and logged as suspicious — basically the same logic real banking apps use: browsing is easy, but a transfer asks you to prove yourself again no matter how long you've been logged in.

Since this is a custom application log, not something Wazuh already understands, a custom decoder had to be built. Getting it working surfaced three separate issues:

- The first version used `offset="after_regex"`, which isn't actually a valid option — only `after_parent` and `after_prematch` are.
- Once that was fixed, the decoder matched but pulled out nothing at all, with no error to explain why. Turned out Wazuh's default pattern engine doesn't understand standard regex like `\S+` — adding `type="pcre2"` to switch engines fixed it.
- Wazuh also quietly renamed the field `"user"` to `"dstuser"` — it's a reserved name internally. The value was always correct, just filed under a different name than expected.

The rule itself (100105) matched fine the moment it was tested in isolation. But it wouldn't fire against real traffic — the application's logging calls were missing a trailing newline, so every log line ran together into one unreadable block. Wazuh reads logs line by line, so only the very first entry in the file was ever visible to it. Adding `\n` to the logging calls and clearing the old file fixed it for good.

6.2 A Session Used From a Different IP

This one goes after the real gap directly — a session created on one machine, then reused from somewhere else, with no login attempt in between. It was tested for real: log in from Debian, grab the session cookie, then replay that exact cookie from Kali straight against the transfer endpoint.

```
curl -c cookies.txt -X POST http://192.168.1.30/securebank/login.php -d
"username=jsmith&password=Winter2026!"
# grab the session ID from cookies.txt, then from Kali:
curl --cookie "PHPSESSID=<stolen_session_id>"
http://192.168.1.30/securebank/transfer.php
```

No login attempt happens on the Kali side at all — just the stolen cookie, straight to the sensitive endpoint. That's what makes this scenario invisible to every rule built in Stage One. The resulting log confirmed the signature: same `session_id`, two different IPs, nothing in between.

Getting the correlation rule (100106) working took more debugging than anything else in this project — six separate issues before it actually worked:

#	Issue	How it got fixed
1	An early draft rule (id 100107) ended up duplicated in the file from repeated edits, and Wazuh rejected the whole ruleset over it	Found with <code>grep</code> , removed with a previewed <code>sed</code> deletion
2	The cleanup left a stray, unclosed XML fragment behind	Spotted by viewing a wider chunk of the file, removed with a second precise deletion
3	<code>if_matched_sid</code> kept saying "Signature ID not found" — the rule it depended on was defined later in the file	Wazuh parses top to bottom, so the dependency had to move earlier — this happened twice and was fixed the same way both times
4	Switching to <code>if_group</code> on a rule with frequency/timeframe caused a hard config error and stopped wazuh-manager from starting at all	Turns out frequency-based rules need the "matched" version of the tag (<code>if_matched_group</code>), not the plain one
5	Even after that fix, <code>same_field</code> and <code>different_field</code> just wouldn't correlate — tried it on the custom <code>session_id</code> field, then on a proper built-in field (<code>dstuser</code>), neither worked	Isolated with a deliberately simple test to rule out the field itself as the problem
6	The actual fix: Wazuh has dedicated tags for this — <code>same_source_ip</code> and <code>same_user</code> — separate from the generic <code>same_field</code> mechanism entirely	Swapping in <code>same_user</code> fixed it immediately, first try after the swap

The rule that finally worked:

```
<rule id="100106" level="12" frequency="2" timeframe="300">
  <if_matched_group>session_event</if_matched_group>
  <same_user />
  <different_field>srcip</different_field>
  <description>Session hijack suspected: same authenticated user active from two
different source IPs within 5 minutes</description>
  <mitre>
    <id>T1550.004</id>
  </mitre>
  <group>attack,session_hijack,</group>
</rule>
```

Rules 100103 and 100105 both carry a shared session_event tag too, so a fresh login, a normal transfer, or a blocked transfer can all feed into this same correlation — any of them can be the second half of the pair that trips the alert.

7. Verifying the Detection

7.1 Stage One

```
Rule: 100101 (level 5) -> SecureBank login failure (fired repeatedly)
Rule: 100102 (level 10) -> Possible credential stuffing attack ... (fired 8 times)
Rule: 100103 (level 3) -> SecureBank login success (fired twice)
Rule: 100104 (level 14) -> Account Takeover suspected ... (fired twice)
```

Both takeover alerts showed up in the Dashboard with the right details — source IP 192.168.1.10 both times, level 14, MITRE T1110 and T1078. They landed about 1.4 seconds apart, which lines up with how fast Hydra actually runs.

Worth noting: Apache's access log records the URL, method, and status code, but not what was actually POSTed — so while two successful logins were confirmed, matching the two real accounts, there's no way to tell from the alert alone which username was involved in which one. Getting that would need application-level logging, which is exactly what Stage Two ended up adding.

7.2 Stage Two

```
Rule: 100105 (level 10) -> Sensitive action performed without recent re-authentication
                          (session step-up missing)
Rule: 100106 (level 12) -> Session hijack suspected: same authenticated user active
                          from two different source IPs within 5 minutes
```

Both fired for real against actual traffic — a genuinely delayed transfer for 100105, and a genuinely stolen and replayed session for 100106 — not just against test input typed into wazuh-logtest.

8. Dashboard

- Account Takeover Alerts — a bar chart on rule.id: 100104, showing a clean spike of 2 right when the attack ran.
- Credential Stuffing Sources — a table on rule.id: 100102, showing one dominant source (192.168.1.10) with 88 stuffing-burst detections.

Both panels are saved together as the ATO Detection Dashboard. Adding panels for 100105 and 100106 is a natural next step.

9. MITRE ATT&CK Mapping

Technique	Name	Tactic	Where it showed up
T1110	Brute Force	Credential Access	Repeated failed logins (100101/100102)
T1110.004	Credential Stuffing	Credential Access	The stuffing burst itself (100102)
T1078	Valid Accounts	Initial Access, Persistence, Privilege Escalation, Defense Evasion	Successful login (100103/100104); sensitive action on a stale session (100105)
T1550.004	Use Alternate Authentication Material: Web Session Cookie	Defense Evasion, Lateral Movement	Same user, two different IPs (100106)

10. Limitations

Both Stage Two signals are inferences, not proof — they tell you a session looks wrong, not who's actually behind it. That's really a limit of any point-in-time authentication system: identity gets checked once, at login, and nothing stops misuse from happening anytime after that. Closing that gap properly would mean continuous authentication or re-verifying at the transaction level, which is well beyond what a log-correlation rule in a SIEM can do. Impossible travel — flagging a login from a new region shortly after activity somewhere else —

is the same idea as rule 100106 with geography swapped in for a raw IP match, and it's a reasonable next step rather than something built here.

11. Recommendations

- Require MFA on every customer-facing login, so a leaked password alone isn't enough.
- Lock or slow down accounts after a handful of failed attempts, so a stuffing run doesn't get far.
- Add velocity and impossible-travel checks — a new IP or region shortly after activity elsewhere — building directly on the same correlation logic used for rule 100106.
- Ask for step-up verification before any sensitive action regardless of session age, rather than relying purely on a timer the way this lab does.
- Rate-limit at the WAF or reverse proxy layer, so a burst of rapid login attempts never even reaches the app.
- Alert on correlated patterns — stuffing plus success, or the same session from two IPs — instead of single-event thresholds, since the combination carries far more weight than either half alone.

12. Lessons Learned

- Default SIEM rules are generic by nature — they can't recognize an application's own login flow or session logic without custom rules written specifically for it.
- Wazuh only lets one rule win per log line. A custom rule meant to refine detection on a given log type has to chain onto whichever built-in rule is actually catching that traffic — confirmed directly with wazuh-logtest, not guessed at.
- Frequency-based rules need at least two matching conditions by design, and need the "matched" version of dependency tags — the plain versions don't turn on Wazuh's internal counting and will throw a hard config error.
- Rule dependencies need to be defined earlier in the file than whatever references them — a forward reference just silently fails to resolve, even though the target rule exists further down.
- Custom decoders need `type="pcre2"` spelled out to use normal regex syntax — Wazuh's default engine fails quietly on patterns it doesn't recognize.
- Certain field names are reserved and get renamed automatically (user becomes dstuser) no matter what you call them in a custom decoder.
- Application logs need a real newline per entry to be read correctly — a rule and decoder can both pass every isolated test and still fail against live traffic if the logging itself runs lines together.
- Generic `same_field/different_field` correlation didn't work reliably here, even on a proper built-in field — the dedicated tags (`same_source_ip`, `same_user`) exist for exactly this and should be the first thing tried, not the last.
- Two weak signals correlated together beat one strong signal alone, every time this was tested — true for the login-based detection and just as true for the session-based detection.

13. Conclusion

This project ended up covering two distinct layers of account takeover detection: catching the attack at the login itself, and then catching what happens after login when the attacker skips that step entirely. The second layer only exists because someone reviewing the first version pointed out a real blind spot, and chasing it down took a lot more debugging than expected — nine separate platform quirks in total, the biggest one being that Wazuh's dedicated correlation tags have to be used in place of its generic ones to get reliable results.

The approach throughout — correlating multiple events instead of trusting any single one, and being upfront about what it can't actually prove — lines up closely with the fraud-detection work Canadian financial institutions are increasingly expected to do under OSFI guidance and the incoming Bank Act changes.